

Introduction to Chemometrics

A Practical Interpretation Guide

Preprocessing · Exploratory analysis · Class modeling (SIMCA) · Predictive modeling · Validation · Model diagnostics · Metrics

Companion reference for the app's full chemometrics platform — spectral preprocessing, exploratory analysis, class modeling, classification and regression, mixture resolution, and model validation and diagnostics. Read it alongside your analysis to understand what each tool does and how to interpret its plots and metrics.

Table of Contents

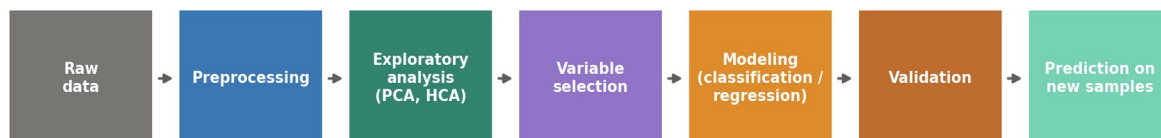
Table of Contents.....	2
1. What is chemometrics?.....	4
2. Signal preprocessing.....	4
2.1 Centering and scaling (universal).....	4
Mean centering.....	4
Autoscaling / Standardization.....	4
2.2 NIR / MIR: scatter correction.....	5
SNV (Standard Normal Variate).....	5
MSC (Multiplicative Scatter Correction).....	5
2.3 EMSC (Extended Multiplicative Scatter Correction).....	6
2.4 Smoothing and derivatives (Savitzky-Golay).....	7
Smoothing (0th-order derivative).....	7
1st derivative.....	7
2nd derivative.....	7
2.5 Raman.....	8
2.6 Chromatograms.....	8
2.7 NMR.....	9
3. Exploratory Data Analysis (EDA).....	10
3.1 PCA (Principal Component Analysis).....	10
Biplot.....	10
Loading plot.....	11
3.2 Outlier detection: Hotelling's T^2 and residual Q.....	11
3.3 HCA — Hierarchical Cluster Analysis.....	12
3.4 t-SNE and UMAP.....	13
3.5 MCR-ALS (Multivariate Curve Resolution — Alternating Least Squares).....	13
4. Variable selection.....	15
4.1 Boruta.....	15
4.2 Genetic Algorithm.....	15
5. Class modeling: SIMCA.....	16
5.1 How each class model is built.....	16
5.2 Why the Q limit must be estimated by cross-validation.....	17
5.3 Acceptance criterion and per-class metrics.....	17
6. Predictive modeling.....	18
6.1 Classification.....	18
6.2 Regression.....	18
6.3 Illustrative diagrams for selected algorithms.....	19
LDA — linear separation.....	19
SVM — maximum margin.....	20
Decision Tree.....	20
Random Forest — voting across many trees.....	21
XGBoost — sequential boosting.....	21
KNN — nearest neighbors.....	21
Neural Network (MLP).....	22
Lasso, Ridge and Elastic Net — regularization.....	22
7. Train/test split and cross-validation.....	23
7.1 Independent test set.....	23
Random split vs. Kennard-Stone.....	23
7.2 Cross-validation (k-fold and Leave-One-Out).....	23

8. Performance metrics.....	25
8.1 Classification.....	25
ROC curve and AUC.....	26
8.2 Regression.....	26
9. Model diagnostics.....	28
9.1 Leverage / Williams plot.....	28
9.2 Learning curves.....	29
9.3 Statistical comparison between models.....	30
10. Hyperparameter optimization.....	32
10.1 Search strategies.....	32
10.2 The app's speed/quality presets.....	32
Putting it all together.....	33

1. What is chemometrics?

Chemometrics applies mathematical and statistical methods to extract chemically relevant information from experimental data — typically spectra (NIR, MIR, Raman, NMR), chromatograms, or other instrumental signals with many variables. Instead of reading a spectrum peak by peak, the full signal is analyzed at once to find patterns, group similar samples, detect anomalies, or predict a property of interest (a concentration, a product class, an origin) from the overall shape of the signal.

A typical chemometrics workflow follows these stages, which also map onto the tabs of the companion app:



Key idea

Every stage depends on the previous one: bad preprocessing ruins PCA; a misread PCA lets outliers slip through; and undetected outliers distort any predictive model built afterward. That is why the order matters, and why this guide follows the same order as the app's tabs.

2. Signal preprocessing

No instrumental signal comes out of the instrument "clean": there is always noise, baseline shifts, or physical measurement effects (light scattering, dilution, alignment) that have nothing to do with the sample's chemistry but still affect the measured signal. Preprocessing aims to correct these unwanted effects without erasing the real chemical information.

2.1 Centering and scaling (universal)

These two transformations are applied to virtually any type of data, as a preliminary step before PCA, PLS, or any other multivariate method — regardless of the type of spectrum.

Mean centering

$$x'_{ij} = x_{ij} - \bar{x}_j$$

For each variable (column) j , the average of that variable across all samples is subtracted. This shifts the origin of the data to the center of the point cloud, so the analysis focuses on the differences between samples (the variance) rather than the absolute signal level. Corrects for: the baseline level common to all samples.

Autoscaling / Standardization

$$x'_{ij} = (x_{ij} - \bar{x}_j) / s_j$$

In addition to centering, each variable is divided by its own standard deviation (computed across samples). The result: every variable ends up with variance 1, carrying the same "weight" in the analysis. Used when variables have very different scales or units, so that zones of higher absolute intensity don't dominate the analysis just because of their magnitude. Corrects for: differences in scale between variables.

2.2 NIR / MIR: scatter correction

In reflectance spectroscopy (typical for NIR on solids and powders), differences in particle size or sample packing create a light-scattering effect that overlays the chemical signal: each spectrum ends up with its own offset and scale, unrelated to chemical composition.

SNV (Standard Normal Variate)

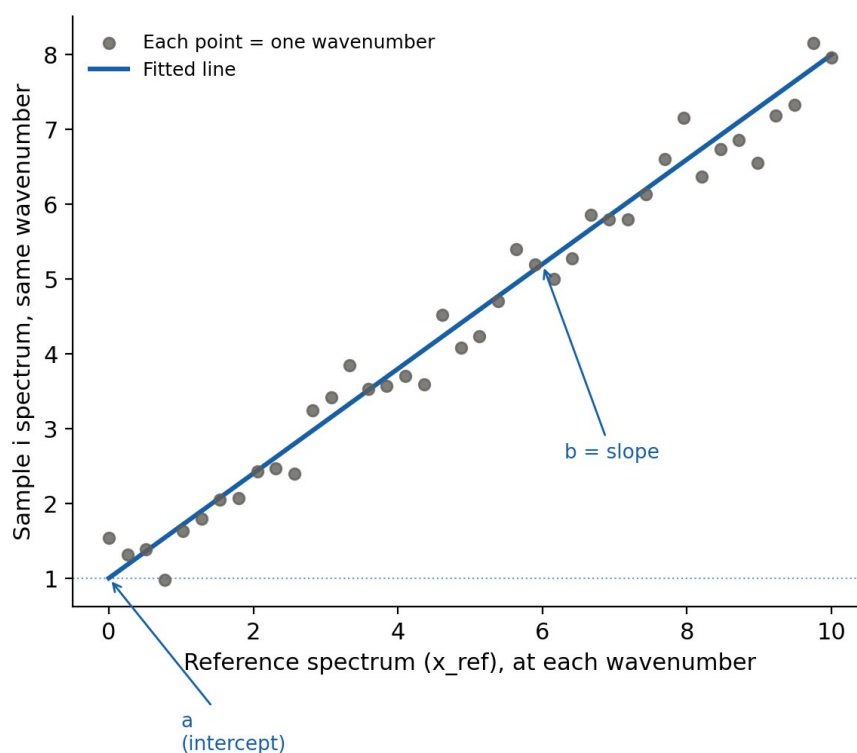
$$x'i = (xi - \bar{xi}) / si$$

For each spectrum (row) i , its own mean is subtracted and it is divided by its own standard deviation, computed across the whole spectrum. Each sample corrects its own scatter independently. Corrects for: light scattering from particle size and optical path differences.

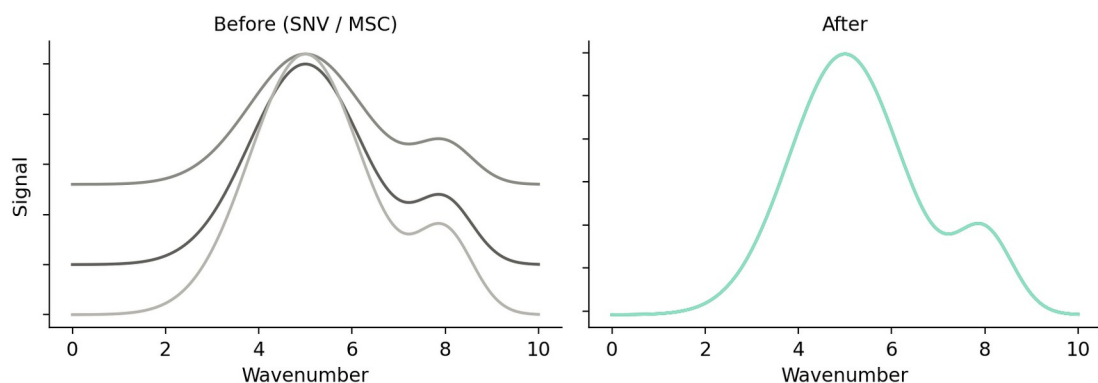
MSC (Multiplicative Scatter Correction)

$$xi = ai + bi \cdot x_{ref} \rightarrow x'i = (xi - ai) / bi$$

Unlike SNV, MSC fits each spectrum against a common reference spectrum (usually the average of all calibration samples) via linear regression. Each point in the plot below is a different wavenumber: on the X axis, how much the reference absorbs there; on the Y axis, how much that sample absorbs at the same point. Without scatter, every point would fall on the ideal diagonal.



a = additive offset · b = scale difference. The correction undoes both effects.



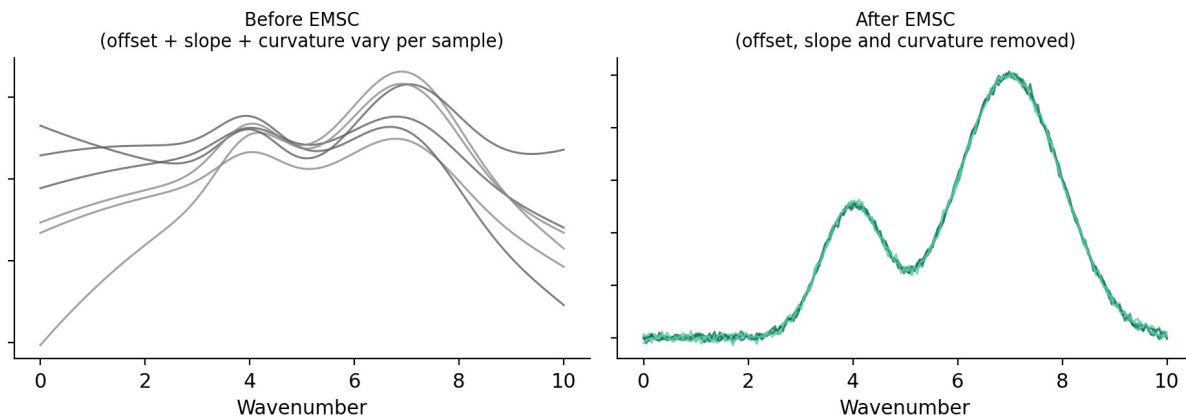
Typical SNV/MSC effect: spectra with different offset and slope collapse to a common shape.

2.3 EMSC (Extended Multiplicative Scatter Correction)

EMSC extends plain MSC by also modeling and removing wavelength-dependent effects — a polynomial in the (normalized) wavelength axis, on top of the usual additive offset and multiplicative scaling. This corrects sloped or curved baseline effects that plain MSC cannot, because MSC assumes the scatter effect is the same at every wavelength.

$$xi(\lambda) = ai + bi \cdot x_ref(\lambda) + ci \cdot p1(\lambda) + di \cdot p2(\lambda) + \dots + ei(\lambda)$$

The correction fits this model per sample by least squares and then removes everything except the $bi \cdot x_ref$ term: $x'i(\lambda) = [xi(\lambda) - ai - ci \cdot p1(\lambda) - di \cdot p2(\lambda) - \dots] / bi$. A polynomial order of 0 behaves like plain MSC; order 2 (the app's default) also removes linear and quadratic baseline slopes, which is the most common real-world scenario.



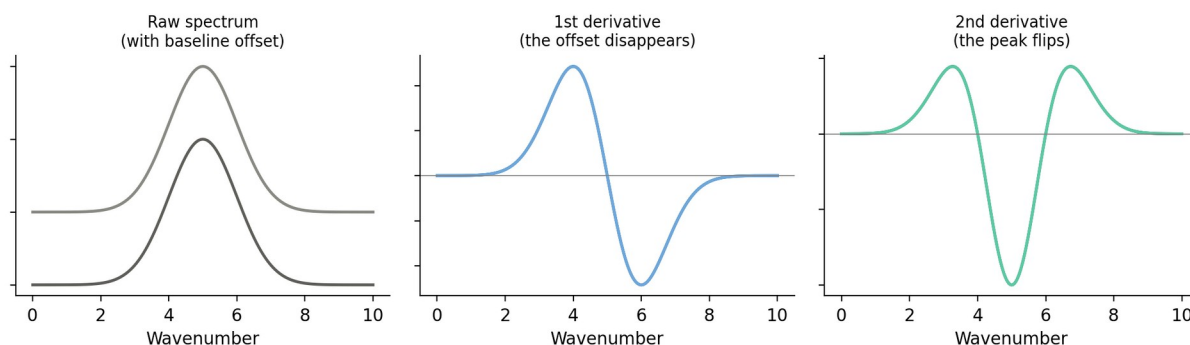
EMSC removes offset, slope AND curvature at once — notice how much flatter and more overlapping the 'after' spectra are compared to plain MSC.

When to prefer EMSC over MSC

If your spectra show scatter effects that clearly depend on wavelength (e.g. a curved or sloped baseline that isn't the same shape across samples), EMSC will usually remove more of that nuisance variation than plain MSC — at the cost of one extra parameter (the polynomial order) to choose. Too high an order risks removing real chemical signal along with the scatter, so start at order 1-2 and compare results.

2.4 Smoothing and derivatives (Savitzky-Golay)

The Savitzky-Golay filter fits a low-order polynomial by least squares within a moving window of points, and replaces the central point with the value of that polynomial (or its derivative). Unlike a simple moving average, it preserves peak shape and width much better.



Smoothing (0th-order derivative)

Corrects for: high-frequency, random instrumental noise.

1st derivative

$$dx / d\lambda$$

Highlights slopes and changes in the spectrum, improving the resolution of overlapping bands. Corrects for: constant (additive) baseline offsets — a spectrum shifted up or down has the same derivative as the original.

2nd derivative

$$d^2x / d\lambda^2$$

Flips peaks (they appear as valleys) and further improves resolution of heavily overlapping bands — widely used for band assignment. Corrects for: constant and sloped (additive and linear) baseline.

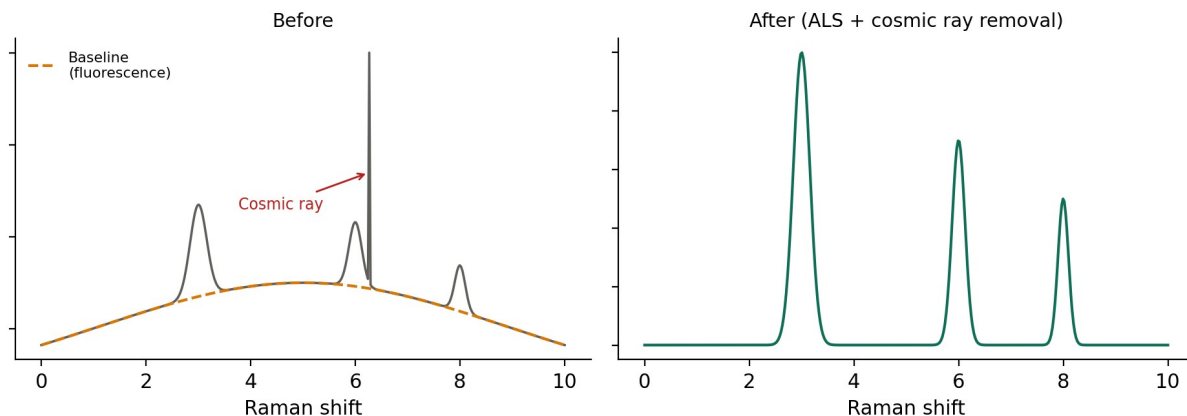
Practical rule

Derivatives amplify high-frequency noise substantially. That is why they are almost always smoothed before differentiating (the Savitzky-Golay filter itself does both in a single pass, given the right window and polynomial order).

2.5 Raman

Raman shares Savitzky-Golay smoothing and derivatives with NIR/MIR, but has two problems of its own that usually dominate its preprocessing:

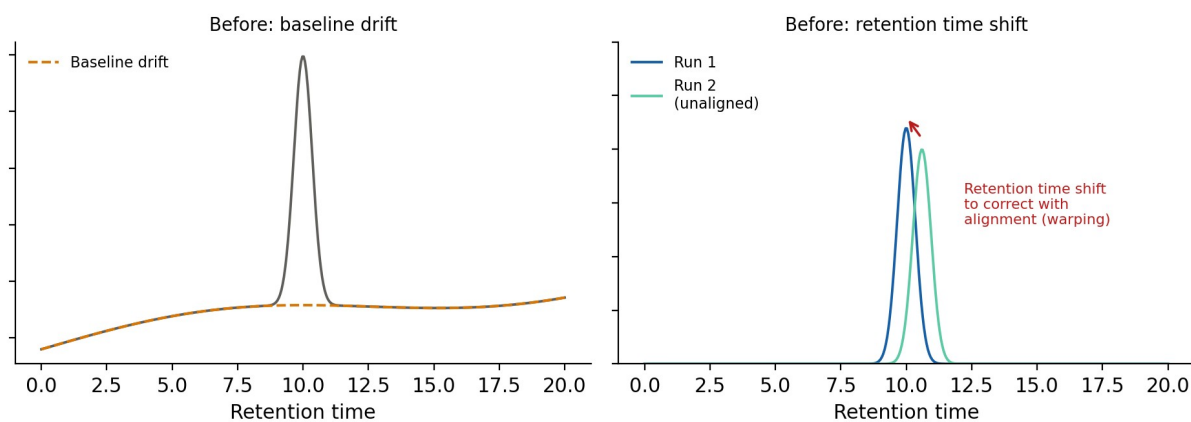
- **Background fluorescence:** a wide, curved baseline, often more intense than the Raman signal itself. Corrected with baseline-fitting methods such as ALS (Asymmetric Least Squares), which fits a smooth curve giving less weight to points above it (the peaks) and subtracts it.
- **Cosmic rays:** narrow, anomalously intense spikes — a CCD detector artifact unrelated to the sample. Detected by comparing each point against a local median filter, and the outlier values replaced.



Left: background fluorescence and a cosmic ray. Right: corrected spectrum.

2.6 Chromatograms

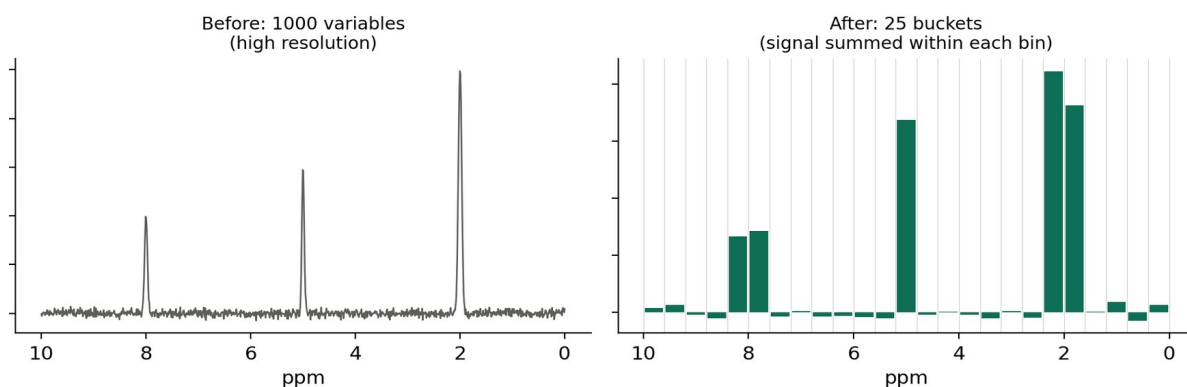
- **Baseline correction:** same concept as in Raman (ALS or other fits), to correct instrumental drift or column bleed.
- **Peak alignment (warping):** techniques such as COW (Correlation Optimized Warping) warp each run's time axis to correct retention-time shifts (from column aging, flow or temperature variations), needed to compare multiple runs against each other.
- **Normalization:** by total area or against an internal standard, to correct for differences in injected volume or sample dilution.



Two typical issues: baseline drift (left) and retention-time shift between runs (right).

2.7 NMR

- **Bucketing / binning:** divides the spectrum into fixed-width "buckets" (e.g. 0.04 ppm) and sums the signal within each, drastically reducing the number of variables. Corrects for: small peak shifts between samples (integrating within a window makes a small shift within that window a non-issue).
- **PQN (Probabilistic Quotient Normalization):** computes a reference spectrum (the median of all samples) and, for each sample, uses the median of the variable-by-variable quotients against that reference as the normalization factor. Corrects for: overall sample dilution differences, more robustly than normalizing by total area.



Bucketing: the high-resolution spectrum is grouped into bins, summing the signal within each.

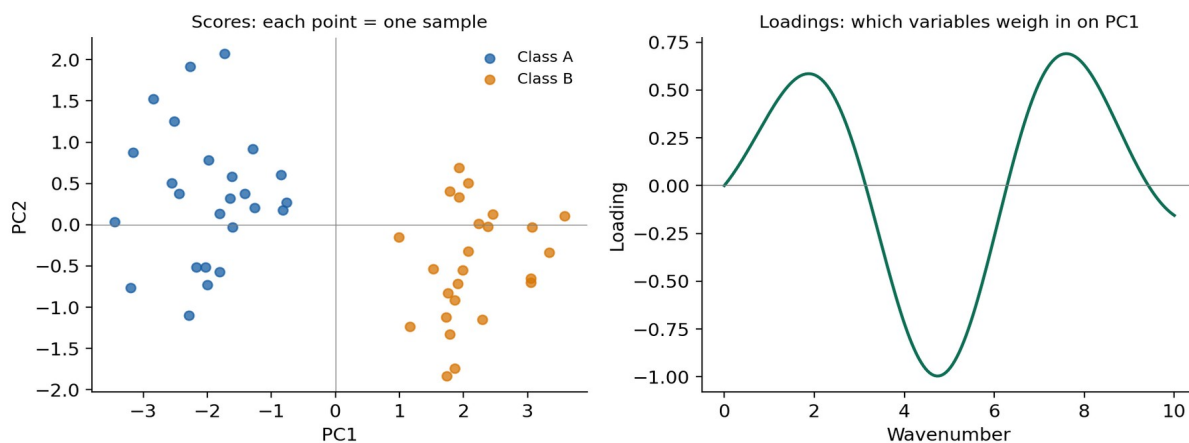
3. Exploratory Data Analysis (EDA)

Before building any predictive model, it helps to "look" at the overall structure of the data: are there natural groupings? Are there unusual samples (outliers) that could distort a model? Exploratory analysis answers these questions without needing a reference variable (class or value to predict).

3.1 PCA (Principal Component Analysis)

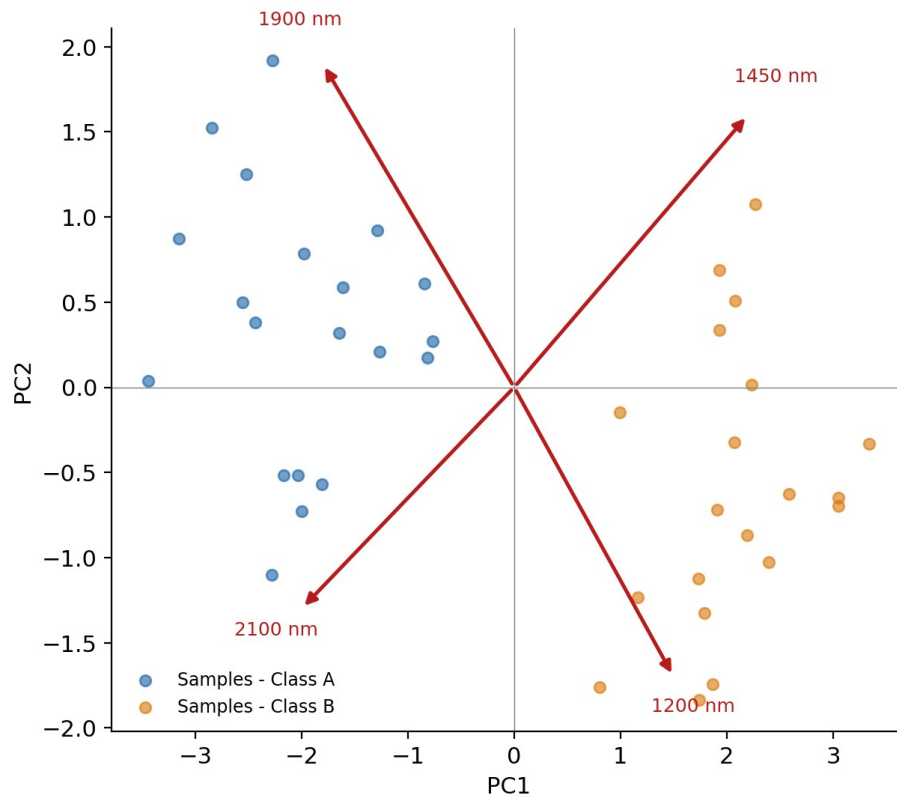
PCA summarizes hundreds of correlated variables (the wavenumbers of a spectrum) into a few "principal components" (PCs) that capture most of the data's variability, in the order in which they explain it. Each component is a linear combination of the original variables.

- **Scores:** each sample's position in the space of the principal components. Chemically similar samples end up close together; different ones, far apart. This is what gets plotted as PC1 vs PC2, etc.
- **Loadings:** how much each original variable (each wavenumber) weighs in on each component. They allow chemical interpretation of which spectral regions explain the observed variability.
- **Explained variance:** the scree plot shows what percentage of the total variability each component captures, helping decide how many are relevant.



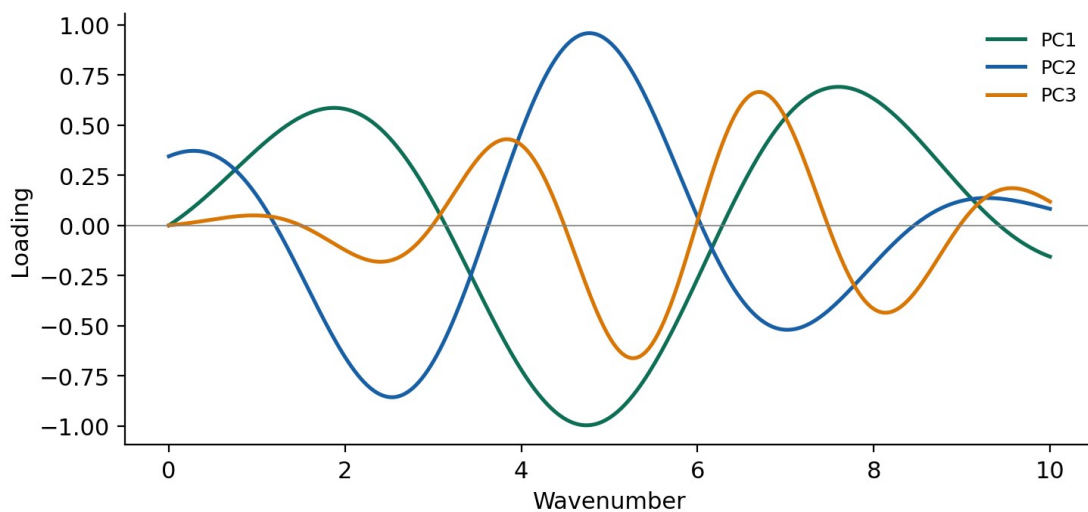
Biplot

Combines scores (the samples) and loadings (the variables) in the same plot, shown as arrows from the origin. The direction and length of each arrow indicates how strongly, and in what direction, that variable influences the plotted components — useful for visually connecting "which samples cluster together" with "which wavenumbers explain it".



Loading plot

Shows the full loading curve of one or more components across the entire spectrum (instead of a single vector, as in the biplot). Lets you pinpoint exactly which wavenumbers each component weighs most heavily, for chemical assignment.



3.2 Outlier detection: Hotelling's T^2 and residual Q

Two complementary statistics are used, computed over a chosen number of components:

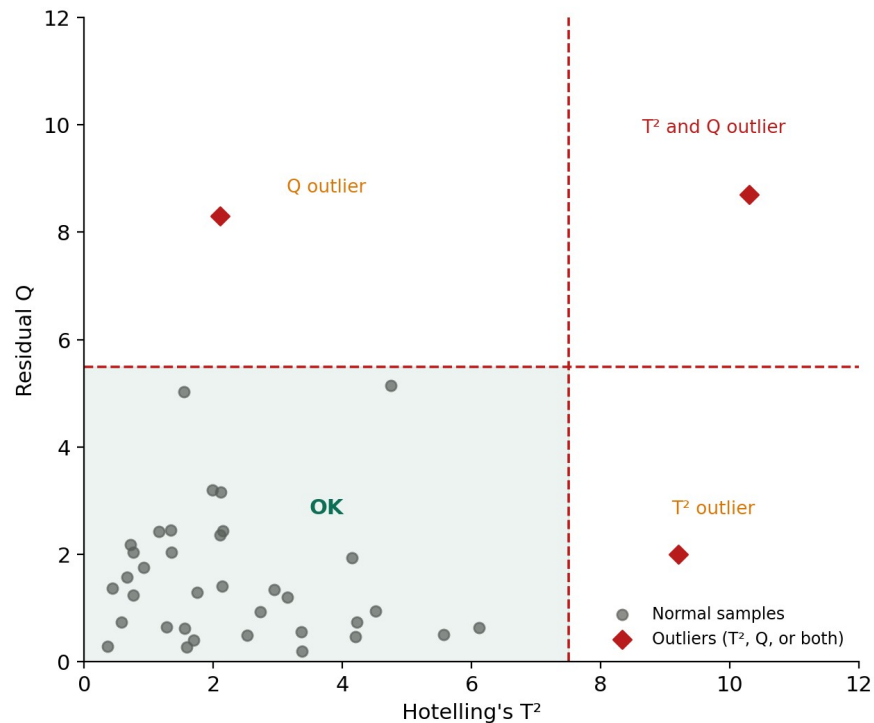
- **Hotelling's T^2 :** measures how far a sample is from the center of the model, within the space defined by the retained components (distance WITHIN the model).

- **Residual Q (SPE):** measures how well each sample is described by the model — the portion of variability that the retained components do NOT capture (distance TO the model).

$$T^2_i = \sum (t_{ik}^2 / \lambda_k) \quad \text{for } k = 1 \dots A \text{ retained components}$$

$$Q_i = \sum (x_{ij} - \hat{x}_{ij})^2 \quad \text{where } \hat{x} \text{ is the model's reconstruction of sample } i$$

A sample can be atypical by T^2 , by Q, or by both at once — each combination points to a different kind of problem.

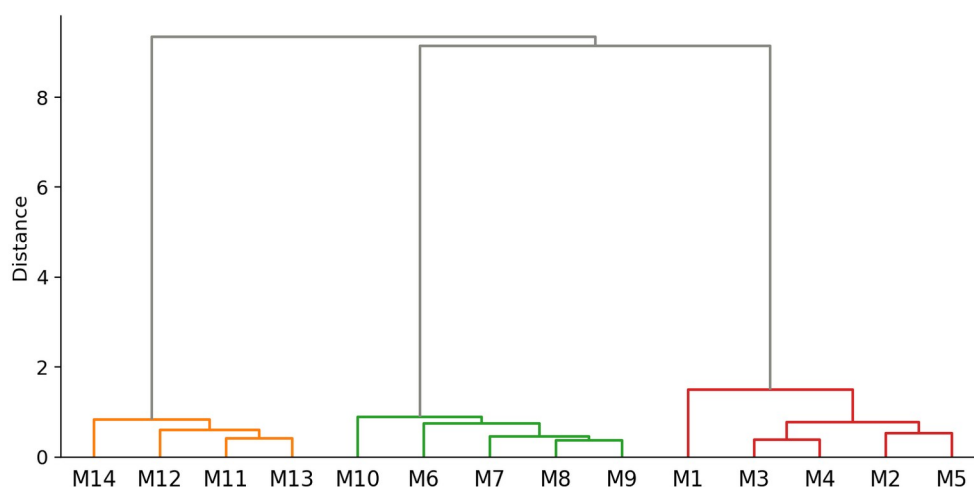


Practical note

The theoretical Q limit (Jackson-Mudholkar approximation) can become unreliable when the retained components already explain nearly 100% of the variance. In those cases, an empirical limit (e.g. the 95th percentile of the observed Q values) is used instead of the theoretical one.

3.3 HCA — Hierarchical Cluster Analysis

Groups samples hierarchically by similarity (Euclidean or other distance), without needing to decide beforehand how many groups exist. The result is visualized as a dendrogram: the lower the point where two samples merge, the more similar they are to each other.



3.4 t-SNE and UMAP

PCA is a linear technique. When classes don't separate well in the space of the first principal components, non-linear techniques such as t-SNE or UMAP sometimes reveal complex groupings better. They don't replace PCA (they're harder to interpret quantitatively, and don't give interpretable loadings), but they are a good exploratory complement.

3.5 MCR-ALS (Multivariate Curve Resolution — Alternating Least Squares)

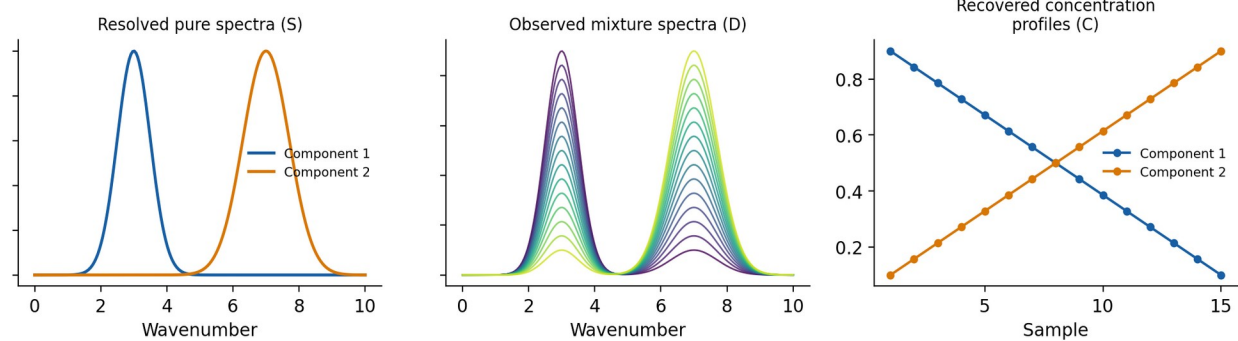
When your samples are MIXTURES, MCR-ALS decomposes the data matrix D into a set of "pure component" spectra (S) and their concentration profile across samples (C), without needing to know the pure spectra beforehand.

$$D \approx C \cdot S + E \quad (\text{samples} \times \text{wavelengths} \approx \text{concentrations} \times \text{pure spectra} + \text{noise})$$

It is solved iteratively: starting from an initial guess (typically from an SVD of D), the algorithm alternates between (1) fixing C and solving for S by least squares, and (2) fixing S and solving for C by least squares — usually under a non-negativity constraint, since real concentrations and spectra can't be negative. This repeats until the reconstruction stops improving.

$$\text{Lack of fit } (\%) = 100 \cdot \text{sqrt}(\Sigma(D - C \cdot S)^2 / \Sigma D^2)$$

Lack of fit measures how much of the data's variance is NOT explained by the resolved components — lower is better; under roughly 5% is generally considered a good fit for clean data. Choosing too few components leaves real mixture behavior unexplained; too many risks splitting real signal into noise-fitting "components" with no chemical meaning.



From left to right: the true pure spectra, the mixture spectra actually measured, and the concentration profiles MCR-ALS recovers from them alone.

4. Variable selection

A spectrum can have hundreds or thousands of wavenumbers, but usually only some regions are truly informative for a specific prediction task. Reducing the number of variables the model uses can improve its performance (less irrelevant noise) and makes chemical interpretation easier.

4.1 Boruta

Compares the importance of each real variable against randomized copies of itself ("shadow variables", generated by shuffling each variable's values across samples at random) using a Random Forest model. A variable is considered relevant only if it consistently outperforms, across many iterations, the best of its random versions. It is a conservative method: it tends to keep everything that contributes real information, even if the contribution is small.

4.2 Genetic Algorithm

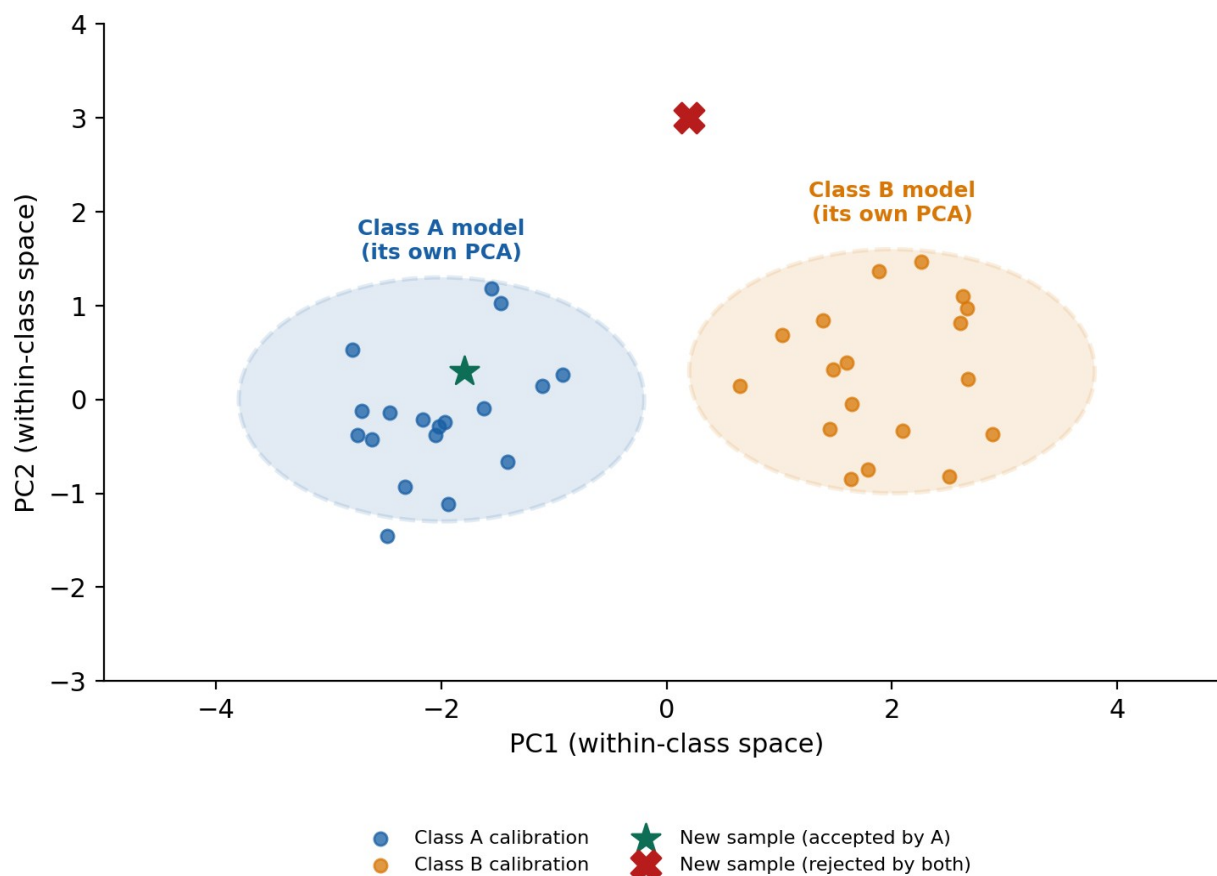
Inspired by biological evolution: each "individual" is a possible subset of variables (encoded as a string of ones and zeros, one per variable). The algorithm evaluates each individual's performance (via cross-validation) and, generation after generation, applies:

- **Selection:** individuals with better performance have a higher chance of "reproducing".
- **Crossover:** the variables chosen by two "parent" individuals are randomly combined to form a "child".
- **Mutation:** with low probability, some variable is switched on or off at random, to avoid getting stuck in a suboptimal solution.
- **Elitism:** the best individual of each generation always survives intact into the next.

After several generations, the best final individual is the proposed variable subset. It is more flexible than Boruta (it can explore specific combinations of variables), but also more computationally expensive and its result depends somewhat more on chance.

5. Class modeling: SIMCA

SIMCA (Soft Independent Modeling of Class Analogies) takes a fundamentally different approach from LDA, PLS-DA, Random Forest, or any other discriminant classifier. A discriminant classifier always forces a pick among the classes it was trained on — even for a sample that doesn't really belong to any of them. SIMCA instead builds one PCA model PER CLASS, and asks, independently for each one, "does this sample look like a member of this class?".



Each class gets its own PCA model (its own ellipse of 'normal' variability). A new sample can be accepted by one class, several, or none.

5.1 How each class model is built

For each class, a PCA is fit using ONLY that class's calibration samples. The number of components is chosen to reach a target cumulative explained variance (e.g. 90%), capped so it never uses more than roughly a third of that class's samples — using too many components relative to sample size leaves almost no residual degrees of freedom, which makes the model fit calibration noise instead of real class structure.

T^2 and Q are then computed exactly as in general outlier detection (section 3.2), but referenced to THAT class's own PCA model — its own components, its own T^2 and Q limits.

5.2 Why the Q limit must be estimated by cross-validation

A subtlety worth understanding: the residual Q computed on the SAME samples used to fit the PCA is systematically optimistic, because PCA is fit specifically to minimize that residual on those samples. Using that in-sample Q directly to set the class boundary makes it too tight — genuinely new samples of the very same class would often get rejected, since their honest residual is larger than what the calibration set showed. The fix is to estimate the Q limit from cross-validated residuals instead: refit the PCA leaving each calibration sample out in turn, project it back in, and use THAT residual (which behaves like a residual on unseen data) to set the boundary. This is what makes a SIMCA class model actually generalize to new samples, rather than only fitting the exact calibration set it was built from.

5.3 Acceptance criterion and per-class metrics

A sample is accepted as a member of a class if it falls within BOTH that class's T^2 limit AND its Q limit. Since every class model is evaluated independently, a sample can end up:

- **Accepted by exactly one class:** a clean call.
- **Accepted by two or more classes:** an ambiguous / overlapping region between classes.
- **Rejected by every class:** a genuine outlier — possibly an unmodeled class, a bad measurement, or something that just doesn't match anything the model has seen.

Two metrics summarize each class model's quality on held-out test samples:

Sensitivity = (true class members correctly accepted) / (all true class members)

Specificity = (non-members correctly rejected) / (all non-members)

Interpreting a SIMCA result

High sensitivity AND high specificity for every class means the classes are well separated and each model generalizes well. Low sensitivity for a class means that class's model is too strict (rejecting its own genuine members) — consider more calibration samples or a lower target variance. A large fraction of ambiguous or outlier samples across the whole test set may indicate genuine overlap between classes, or that the classes as defined don't correspond to a clean chemical distinction.

6. Predictive modeling

Once the data is preprocessed and (optionally) reduced to the most relevant variables, a model can be trained to predict a class (classification) or a continuous numeric value (regression) from the spectrum.

6.1 Classification

Algorithm	Core idea
LDA	Finds the linear combination of variables that best separates the classes, maximizing the distance between their centers relative to their internal spread.
PLS-DA	PLS regression on the classes coded as dummy variables; classifies according to the class with the highest predicted value. Widely used in chemometrics because it handles many correlated variables well.
Logistic Regression	Models the probability of belonging to each class via a logistic function over a linear combination of the variables.
Naïve Bayes	Applies Bayes' theorem assuming independence between variables; simple and fast, works surprisingly well in many cases.
Random Forest	Combines many decision trees trained on random subsets of data and variables, averaging their votes.
SVM	Finds the boundary (hyperplane) that separates the classes leaving the widest possible margin; with non-linear kernels it can separate curved boundaries.
Decision Tree	Splits the variable space with a sequence of simple if-then rules, easy to interpret.
XGBoost	Builds decision trees sequentially, each one correcting the previous one's errors (gradient boosting); very competitive performance.
KNN	Classifies each sample according to the majority class among its k nearest neighbors in variable space.
Neural Network (MLP)	Layers of connected artificial neurons, able to learn complex non-linear relationships between variables and classes.

6.2 Regression

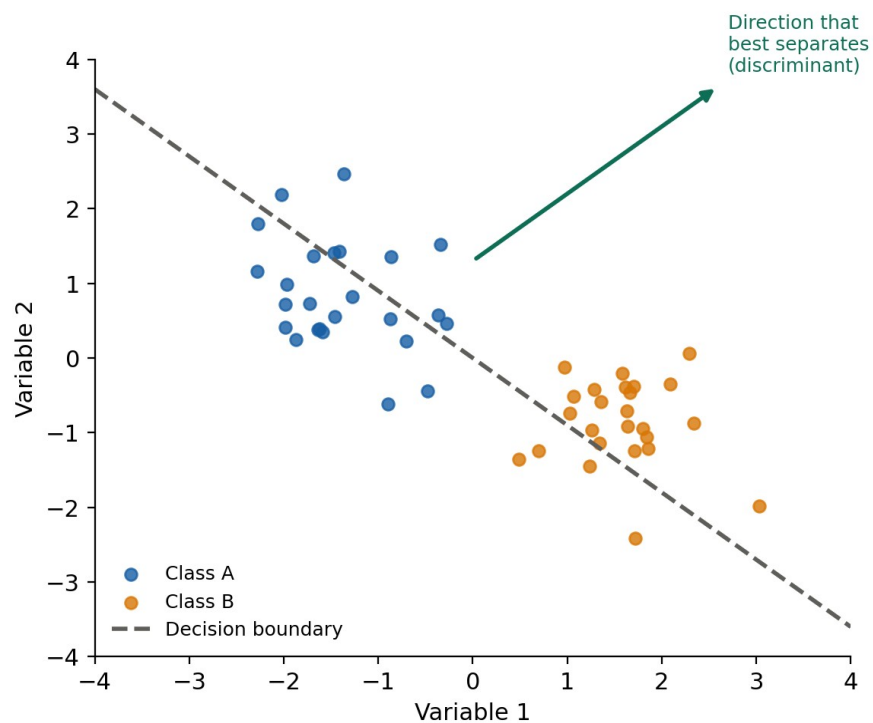
Algorithm	Core idea
PLS	The chemometrics standard: finds components that maximize the covariance between the spectrum and the variable to predict (not just the spectrum's own variance, like PCA).
PCR	Principal Component Regression: PCA first, then a linear regression on the scores.
MLR	Classic multiple linear regression; performs poorly with many correlated variables unless they've already been reduced.
Lasso	Linear regression with a penalty that can push coefficients exactly to zero — performs

Algorithm	Core idea
	variable selection automatically.
Ridge	Linear regression with a penalty that shrinks (without zeroing) coefficients, useful when many variables are correlated.
Elastic Net	Combines Lasso's and Ridge's penalties.
Random Forest / SVM / Decision Tree / XGBoost / KNN	Same algorithms as in classification, adapted to predict a continuous numeric value instead of a class.
Neural Network (MLP)	Same as in classification, adapted for continuous numeric output.

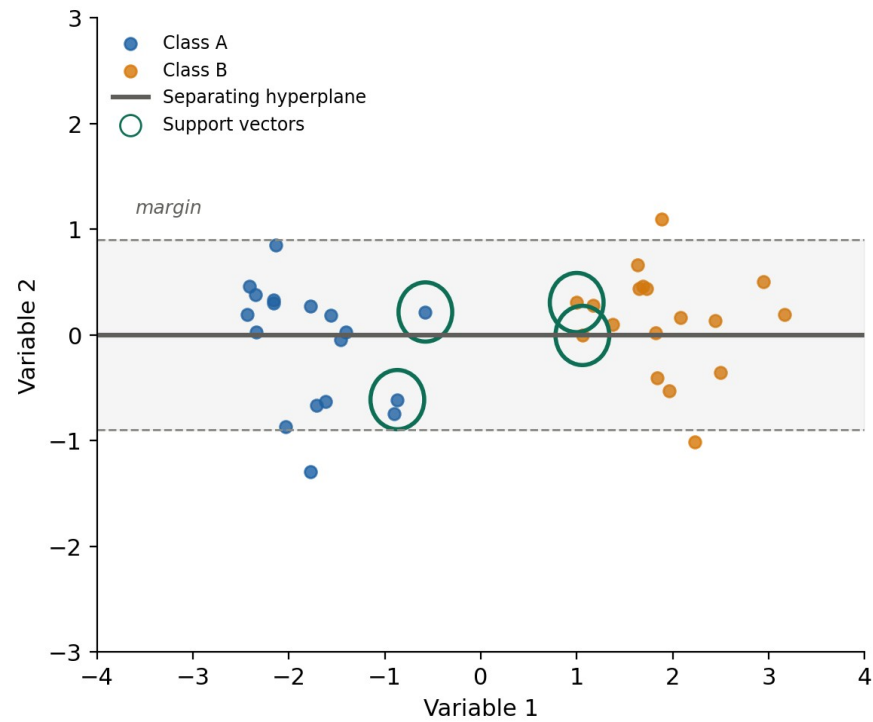
6.3 Illustrative diagrams for selected algorithms

Not all 21 algorithms are illustrated (several share the same visual logic between classification and regression), but these diagrams help build intuition for each family.

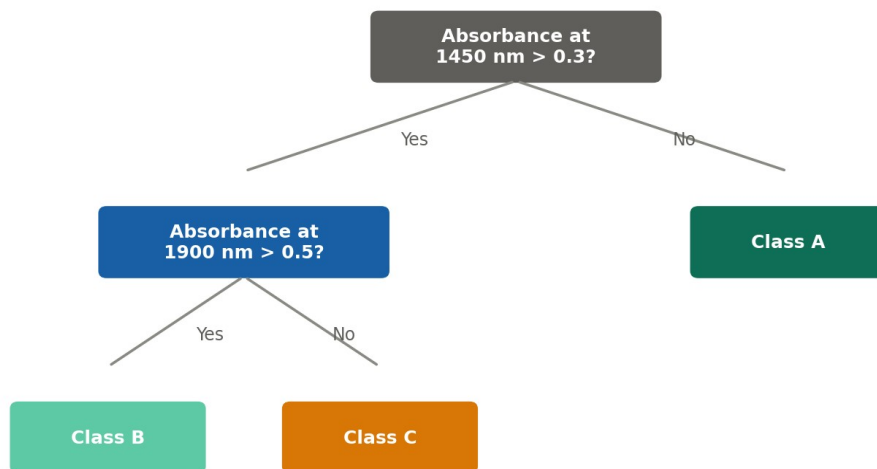
LDA — linear separation



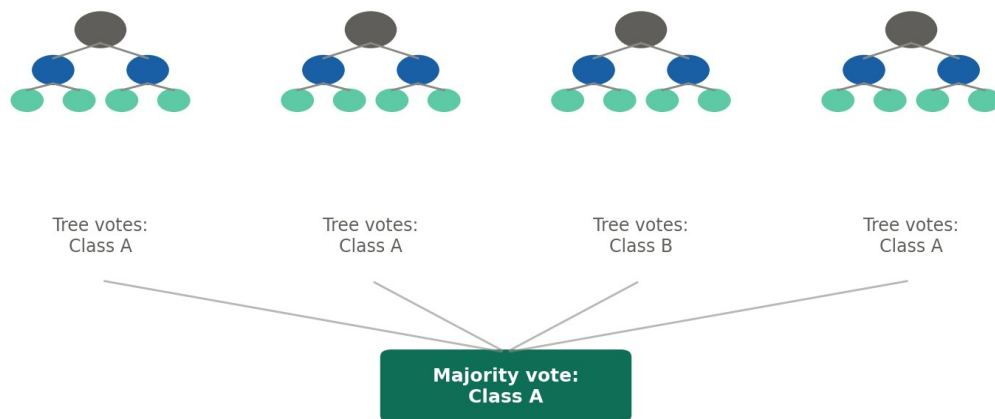
SVM — maximum margin



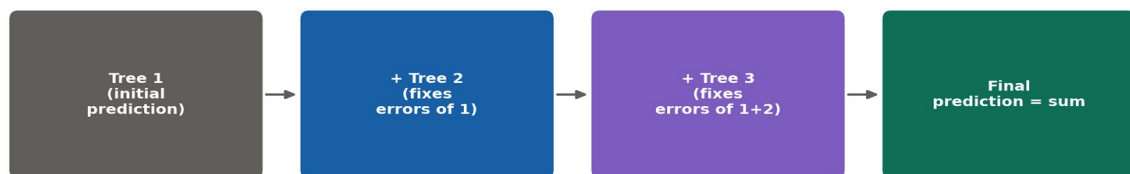
Decision Tree



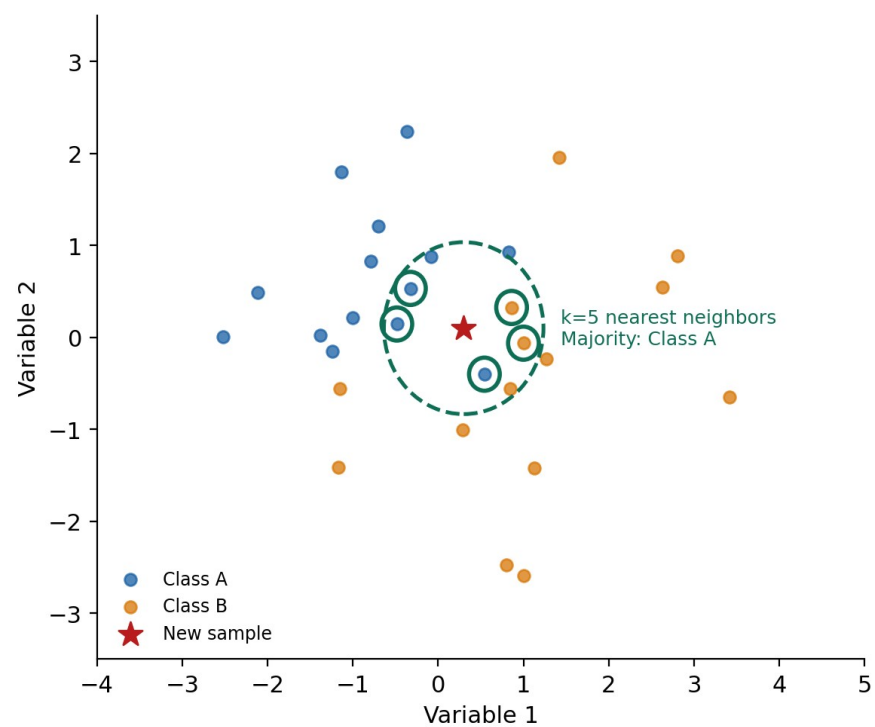
Random Forest — voting across many trees



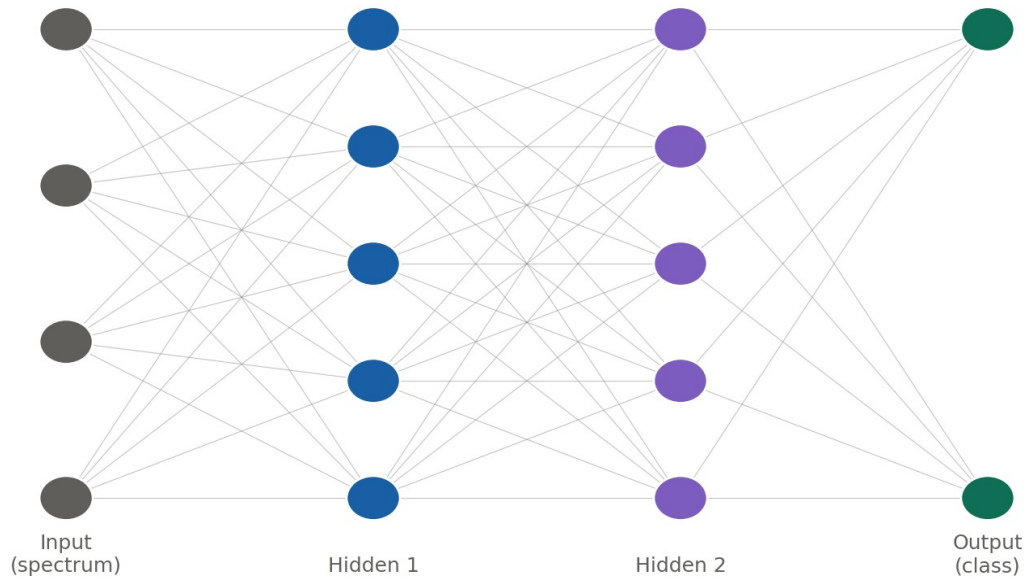
XGBoost — sequential boosting



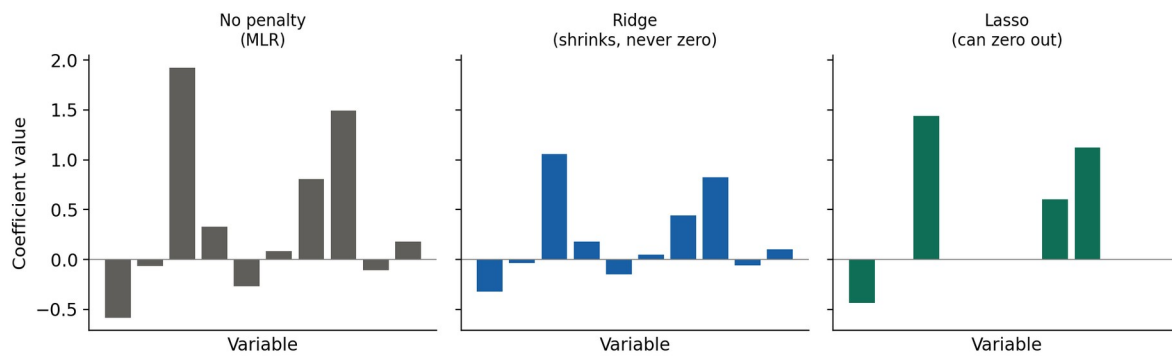
KNN — nearest neighbors



Neural Network (MLP)

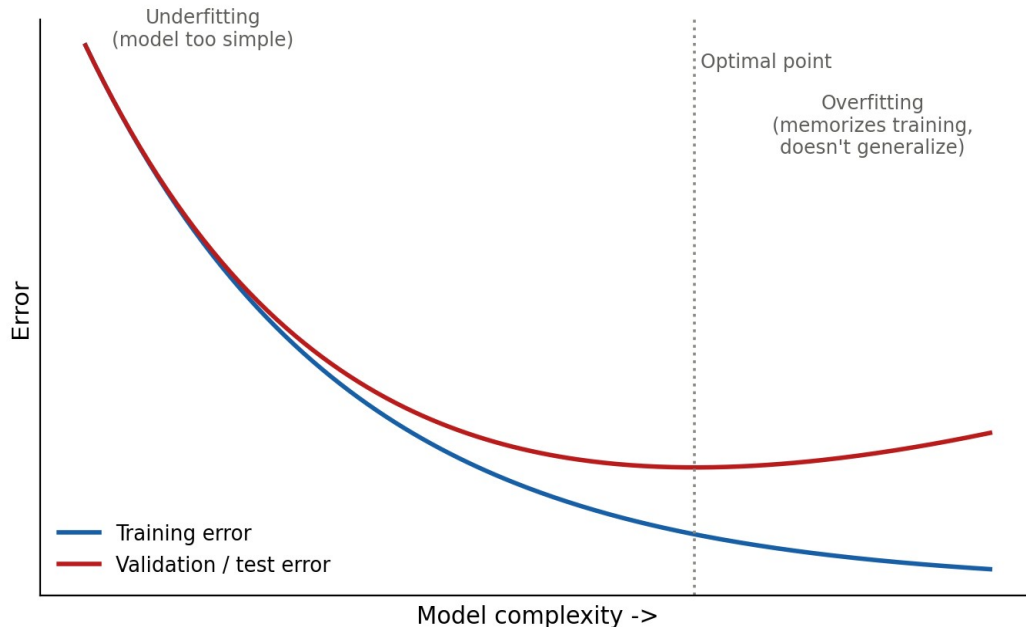


Lasso, Ridge and Elastic Net — regularization



7. Train/test split and cross-validation

A model will always fit the data it was trained on better than it fits new data — the relevant question is how much of that good fit is real learning versus "memorizing" quirks of the training set (overfitting). That is why a model is never evaluated with the same data it was trained on.



7.1 Independent test set

Before training anything, a portion of the samples (e.g. 20%) is set aside and never seen during development. It serves as one final, honest estimate of how the model would perform on truly new samples.

Random split vs. Kennard-Stone

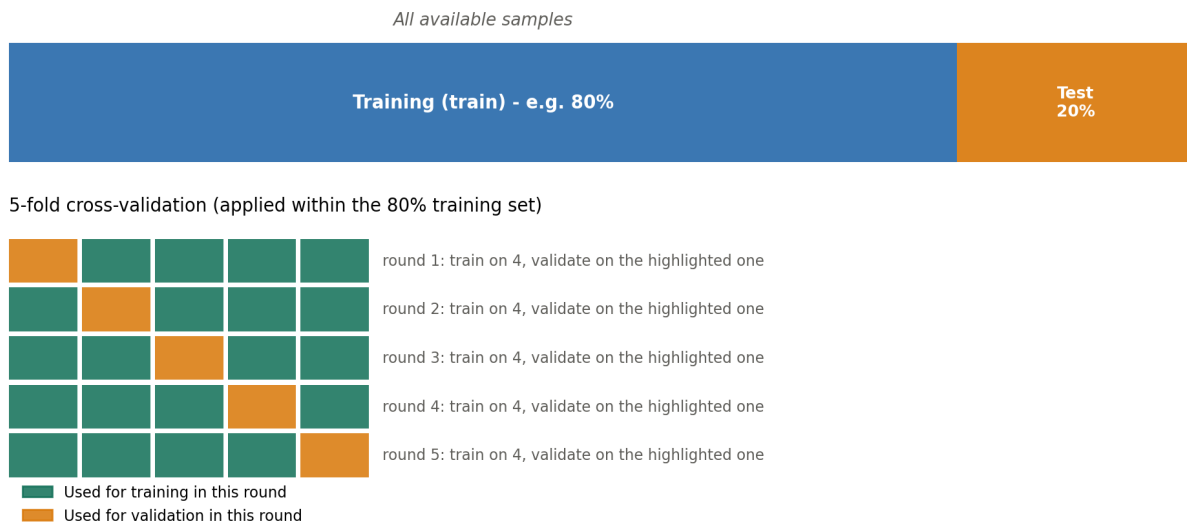
The default way to choose the test set is a random (stratified) split. An alternative, common in chemometrics, is the Kennard-Stone algorithm: it iteratively selects the sample farthest (in Euclidean distance) from the ones already selected, so the chosen TRAINING set broadly covers the X-space, including the more "extreme" samples. The remaining, more "typical" samples become the test set. This tends to give a more stable, representative test estimate than a purely random split, especially with small datasets — though it is not stratified by class, so it's worth checking the resulting class balance.

7.2 Cross-validation (k-fold and Leave-One-Out)

Within the training set, k-fold cross-validation splits the data into k equal parts; over k rounds, the model trains on k-1 parts and validates on the remaining one, rotating which part is left out. In the end, every sample was used exactly once for validation — giving a more stable performance estimate than a single split, without spending extra samples that could be used for training. Typical values: 3, 5, or 10 folds.

Leave-One-Out (LOO) is the extreme case: k equals the number of samples, so each round leaves out and predicts exactly ONE sample, training on all the rest. It is the most thorough option (every sample gets its own

individual held-out prediction) and the standard choice for very small datasets, but it is also the slowest, since it trains one model per sample.



Why not always use a very high k (or LOO)?

With more folds, each model trains on more data and the estimate has less bias, but the process is slower and the estimate can have more variance between folds if there are few samples per class. With fewer folds it's faster, but each model trains on less data. There is no universally correct value — it depends on the size of the dataset.

8. Performance metrics

8.1 Classification

True class	A	14	1	0
	B	2	12	1
	C	0	1	13
		A	B	C
		Predicted class		

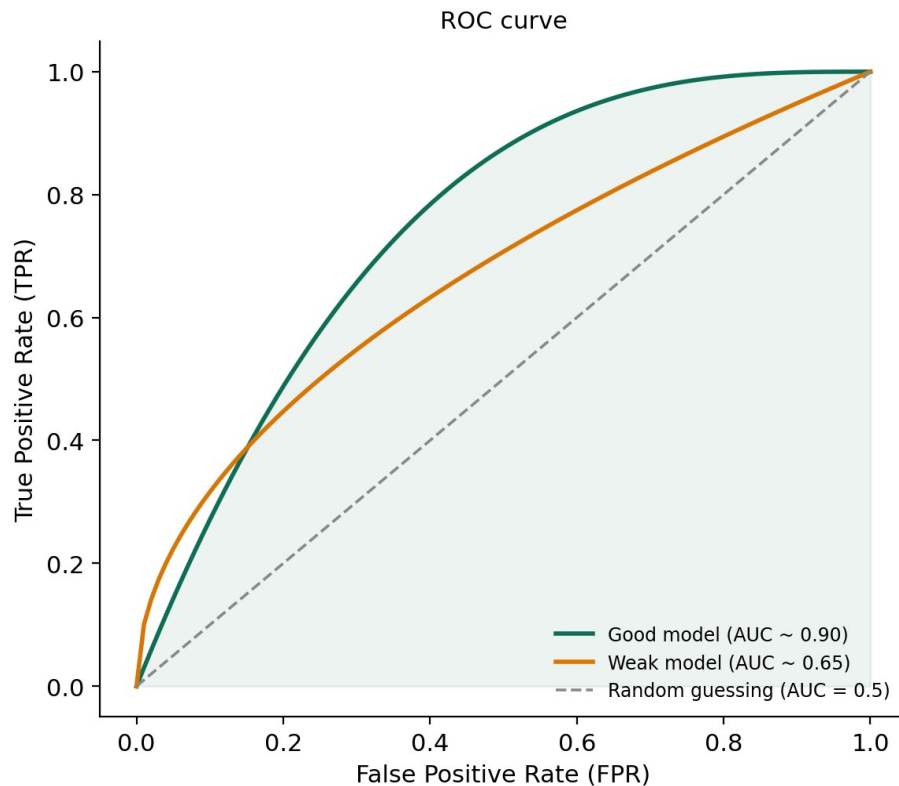
Confusion matrix: the diagonal is correct predictions; off-diagonal, the errors (and which class they were confused with).

Metric	What it measures
Accuracy	Overall proportion of correct predictions. Misleading if classes are imbalanced.
Balanced Accuracy	Average of the per-class accuracy, computed separately for each class — not fooled by classes with many more samples than others.
Sensitivity (Recall)	Of everything that truly IS class X, how much did the model detect? Formula: $\text{Sensitivity} = \text{TP} / (\text{TP} + \text{FN})$.
Specificity	Of everything that truly is NOT class X, how much did the model correctly rule out? Formula: $\text{Specificity} = \text{TN} / (\text{TN} + \text{FP})$.
Precision	Of everything the model labeled as class X, how much really was. Formula: $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$.
F1 (macro)	Average, across classes, of the harmonic mean between precision and sensitivity: $\text{F1} = 2 \cdot \text{Precision} \cdot \text{Recall} / (\text{Precision} + \text{Recall})$.
Cohen's Kappa	How far above chance the model's agreement with the truth is (0 = like flipping a coin; 1 = perfect). Accounts for the accuracy the model would get purely by chance given the class proportions.
MCC (Matthews Correlation Coefficient)	A single balanced measure that uses all four confusion-matrix quadrants at once, robust to class imbalance. Ranges from -1 (total disagreement) to +1 (perfect); 0 means no better than random.
AUC (Area Under the ROC Curve)	Summarizes how well the model ranks samples by predicted probability of belonging to a class, independent of any single decision threshold. 1.0 = perfect ranking; 0.5 = random.

$$MCC = (TP \cdot TN - FP \cdot FN) / \sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}$$

ROC curve and AUC

The ROC (Receiver Operating Characteristic) curve plots the True Positive Rate (sensitivity) against the False Positive Rate (1 - specificity) as the classification threshold is swept across every possible value. A model that ranks samples well traces a curve that rises quickly toward the top-left corner; a model no better than random guessing traces the diagonal line. The Area Under the Curve (AUC) condenses the whole curve into a single number: the probability that the model ranks a randomly chosen positive sample above a randomly chosen negative one.



A better model (green) sits closer to the top-left corner and has a larger area under its curve than a weaker one (orange). The diagonal represents random guessing.

8.2 Regression

Metric	What it measures
RMSE	Root mean squared error — the model's typical prediction error, in the same units as the Y variable.
MAE	Mean absolute error — similar to RMSE, but less sensitive to occasional large errors.
R ²	Proportion of Y's variability explained by the model (1 = perfect fit; 0 = same as always predicting the average).
RPD	Standard deviation of Y divided by RMSE. Common NIR rule of thumb: RPD > 2 acceptable, > 3 good.

Metric	What it measures
RPIQ	Robust version of RPD: uses Y's interquartile range instead of its standard deviation (less sensitive to outliers in Y).

9. Model diagnostics

Beyond the headline metrics, a few extra diagnostic tools help judge whether a model is trustworthy, whether it would benefit from more data, and whether an apparent improvement over another model is real or just noise.

9.1 Leverage / Williams plot

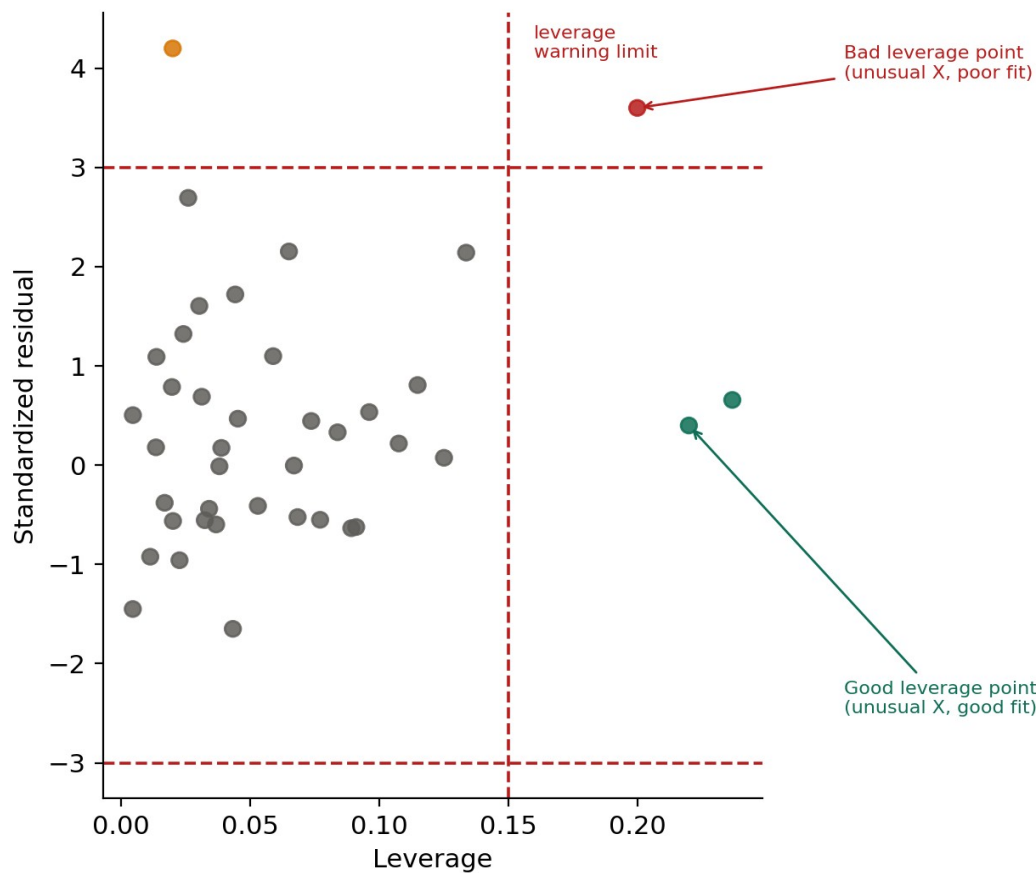
A classic applicability-domain diagnostic, very common in QSAR and chemometrics. Leverage measures how far a sample sits from the center of the spectral space — a generic, algorithm-independent measure computed from a PCA fit on X, so it can be paired with the prediction residual of ANY regression model.

$$h_i = 1/n + \sum_a (t_{ia}^2 / \sum_j t_{ja}^2) \quad (\text{computed from PCA scores } t \text{ of } X)$$

$$\text{Standardized residual} = (y_i - \hat{y}_i) / \text{std}(y - \hat{y})$$

Plotting leverage (X-axis) against the standardized residual (Y-axis) — the Williams plot — separates samples into four regions:

- **Normal samples:** low leverage, small residual — well within the model's comfort zone.
- **Good leverage points:** high leverage but small residual — an unusual spectrum, but the model still predicts it well. These extend the calibration range usefully.
- **High residual, normal leverage:** a typical-looking spectrum that the model still predicts poorly — worth checking the reference value for that sample.
- **Bad leverage points:** high leverage AND a large residual — an unusual spectrum that the model also predicts poorly. These are the samples most likely to be distorting the model and deserve a closer look (measurement error? a genuinely different sample type?).

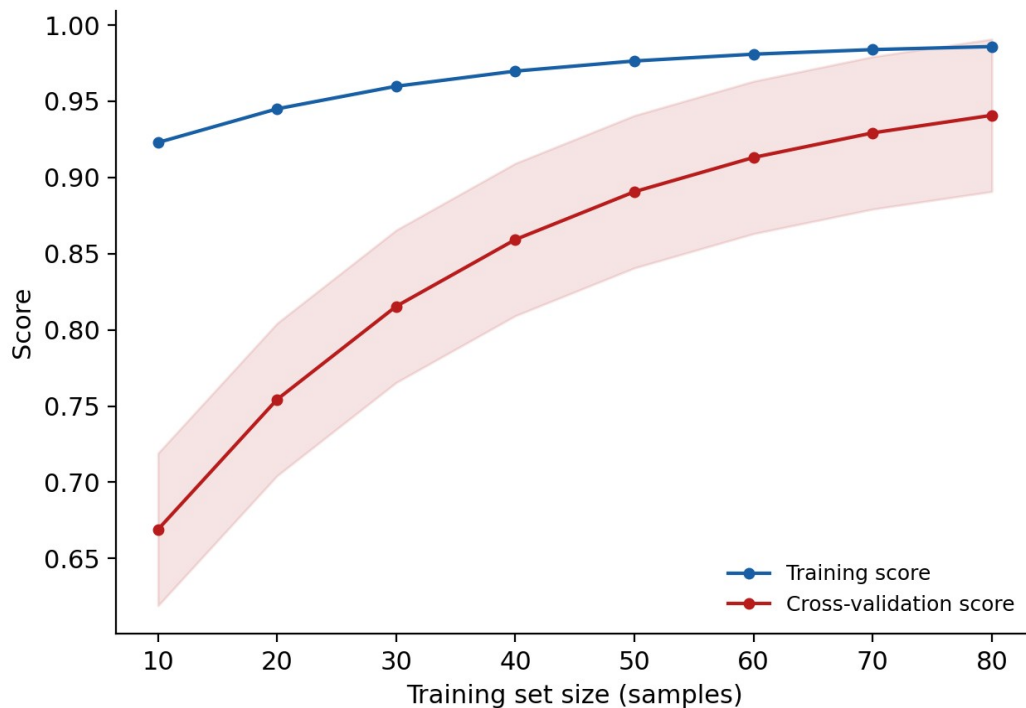


$$h^* = 3 \cdot (A + 1) / n \quad (\text{standard leverage warning threshold; } A = \text{components, } n = \text{samples})$$

9.2 Learning curves

A learning curve plots training and cross-validation performance as a function of how many samples were used to train the model. It helps diagnose whether the model would benefit from more data:

- **Both curves still rising and far apart:** more samples would likely help — the model hasn't seen enough data yet.
- **Curves flat and close together:** the model has plateaued; more samples of the same kind probably won't improve it much. A different algorithm, more variables, or better preprocessing may be needed instead.

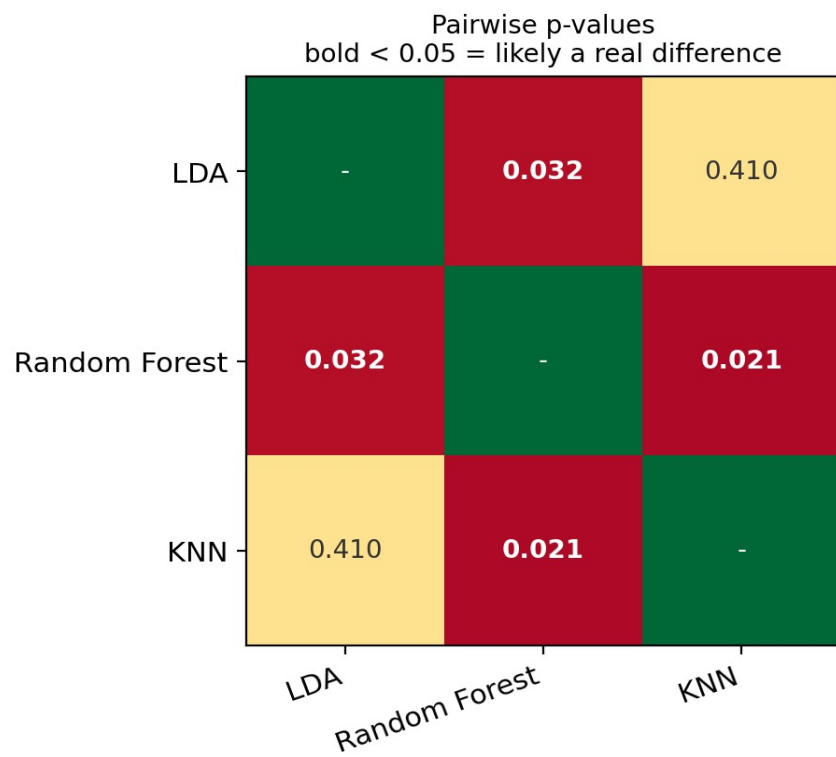


9.3 Statistical comparison between models

A model comparison table often shows one model scoring numerically higher than another — but is that difference real, or could it just be noise from this particular split into folds or this particular test set? A formal statistical test answers that question directly, and the right test depends on whether an independent test set is available:

- **With a held-out test set (preferred):** for classification, McNemar's test compares two classifiers' predictions on the SAME test set, sample by sample — it looks only at the samples where they disagreed, and checks whether one of them was wrong noticeably more often on those. For regression, a paired t-test on the per-sample squared errors of the two models serves the same purpose. Both can genuinely detect overfitting, since a model that overfits will show larger test errors even if its cross-validation score looked fine.
- **Without a test set (cross-validation only):** a paired t-test is run on the per-fold cross-validation scores of the two models (using the exact same folds for both, for a fair comparison). This cannot detect overfitting the way a true held-out test can, since cross-validation folds still influence which samples contributed to the final model choice.

A p-value below 0.05 is the usual convention for calling a difference statistically significant — meaning it's unlikely to be due to chance alone, given this data.



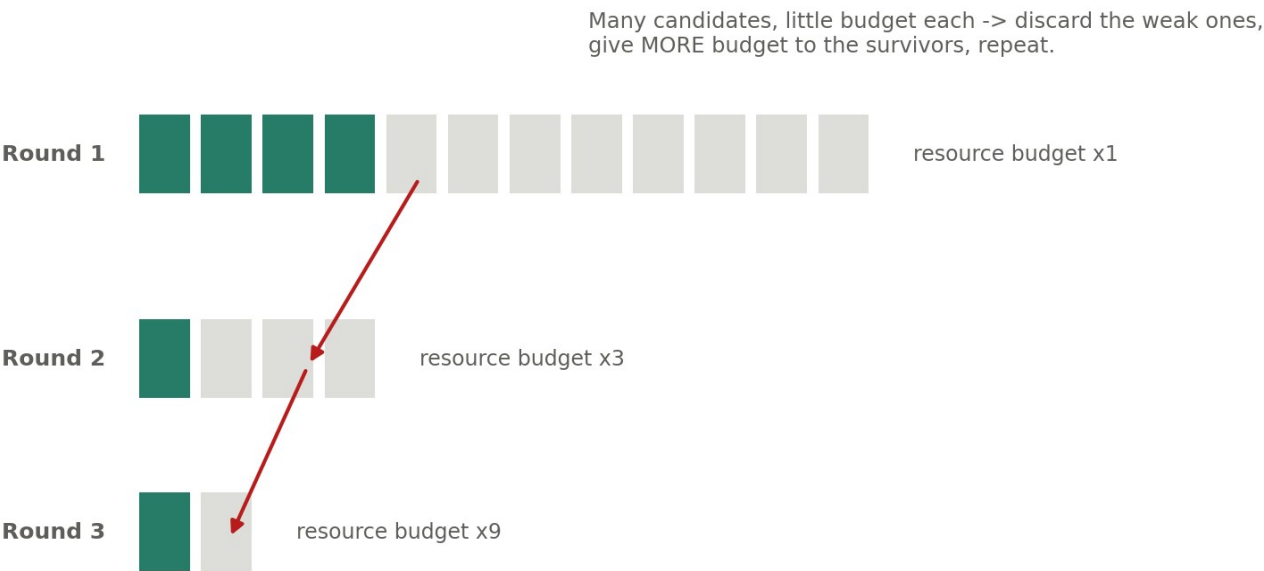
Example output: Random Forest and KNN differ significantly ($p=0.021$, bold); LDA is not significantly different from either.

10. Hyperparameter optimization

Many algorithms have settings of their own that aren't learned from the data automatically (for example, how many trees to use in a Random Forest, or how many latent variables to keep in a PLS model). Choosing these "hyperparameters" well can meaningfully improve a model's performance.

10.1 Search strategies

- **Grid search:** exhaustively tries every combination of a list of candidate values for each hyperparameter, evaluating each combination by cross-validation. Guarantees finding the best combination within the grid, but can be slow with many hyperparameters or candidate values.
- **Random search:** instead of trying every combination, tries a fixed number of randomly chosen combinations. Usually finds a combination almost as good as the full grid, in much less time.
- **Successive halving:** starts by giving EVERY candidate combination only a small amount of resources (few training samples, or few trees) — enough for a rough estimate. It then discards the weaker half (or more) and gives the survivors a bigger resource budget, repeating until only the best combinations remain, now evaluated with the full budget. This avoids ever spending full computation on combinations that were clearly going to perform badly.



Successive halving: many candidates start with a small budget; only the promising ones survive to receive a larger budget in the next round.

10.2 The app's speed/quality presets

Rather than choosing a raw strategy directly, the app offers three presets that map onto the strategies above:

Preset	What it runs	Best for
Fast	Random search, 8 combinations tried.	A quick first pass, or when you're iterating on

Preset	What it runs	Best for
		preprocessing choices and don't need the best possible model yet.
Balanced (default)	Successive halving from random starting combinations.	The best speed/quality trade-off for most day-to-day use.
Thorough	Successive halving over the FULL grid.	Still checks every combination in the grid, just far more efficiently than classic grid search — use when you want the most complete search you can afford.

For Random Forest and XGBoost, successive halving grows the NUMBER OF TREES across rounds (since that is the natural "resource" for those models); for every other algorithm, it grows the number of TRAINING SAMPLES used instead. With very few samples, or with Leave-One-Out cross-validation combined with a sample-based resource, the app automatically falls back to a plain grid/random search, since successive halving's resource-scheduling assumptions don't hold in those edge cases.

Important

Hyperparameter optimization must always be done using cross-validation on the training set, never by looking at the result on the test set — otherwise the test set stops being an honest estimate of performance on new data.

Putting it all together

This guide accompanies the app's full chemometrics platform — far more than an exploratory tool: every concept explained here has a matching control somewhere in the app (choosing a preprocessing step, viewing PCA, training a classification or regression model, building a SIMCA class model, running MCR-ALS, checking a Williams plot or a learning curve, comparing models statistically...). The best way to internalize these concepts is to try them on real data, watching how the results change as each setting is adjusted — and to come back to this guide whenever a plot or a metric needs a second look.